

X5 — El cerebro de Alginvesting

Surrogate model para el ajuste dinámico de parámetros de trading

Versión extendida — guía técnica y conceptual completa

Documento de referencia para entender X5 de principio a fin: qué hace, por qué, cómo está construido (Gradient Boosting y FT-Transformer), cómo se entrena desde cero con backtesting paralelo, cómo recomienda los parámetros óptimos y cómo se integra al sistema. Explicado con analogías y ejemplos numéricos, pensado para leerse impreso sin necesidad de consultar el código.

Índice

1	Introducción: qué es X5 y por qué es el cerebro	3
1.1	El sistema en 30 segundos	3
1.2	Los parámetros que X5 controla	3
1.3	El flujo completo	4
2	El problema que resuelve X5	5
2.1	Por qué los parámetros fijos no bastan	5
2.2	El enfoque: surrogate model + optimización en inferencia	5
3	La analogía del chef (el hilo conductor)	6
4	Los datos: qué entra y qué sale del modelo	7
4.1	Panorama de los inputs (~240 columns)	7
4.1.1	Bloque 1 y 2 — Contexto de mercado (X2 y X3), en dos momentos	7
4.1.2	Bloque 3 — Features temporales	8
4.1.3	Bloque 4 — Parámetros de configuración	8
4.1.4	Bloque 5 — Estado del portfolio	8
4.2	Los outputs: 3 variables objetivo	8
5	El store: el cuaderno de recetas	9
5.1	Dos tipos de fila	9
5.2	Por qué son necesarios los dos tipos	9
5.3	Schema real implementado	9
6	Arquitectura que escala con los datos	11
6.1	Tres niveles según cuántos datos hay	11
6.2	Estado actual	11
7	V1 — Gradient Boosting (LightGBM)	12
7.1	La intuición: un comité que corrige errores	12
7.2	Cómo se ve en el código	12
7.3	Ventajas y el único contra	13
8	V2 — FT-Transformer (Feature Tokenizer + Transformer)	14
8.1	Paso 1 — Feature Tokenizer: cada número se vuelve un vector	14
8.2	Paso 2 — Atención: las features se hablan entre sí	14
8.2.1	El problema que resuelve la atención	14
8.2.2	La analogía de la reunión de equipo	15
8.2.3	El mecanismo concreto: Query, Key, Value	15
8.2.4	Multi-head: mirar desde varios ángulos	16
8.3	Paso 3 — Apilar capas: de lo simple a lo abstracto	16
8.4	Dentro de cada bloque Transformer	17
8.5	La salida multi-head: predecir 3 cosas a la vez	18
9	Inferencia: encontrar los parámetros óptimos	20
9.1	Con LightGBM: búsqueda con Optuna	20
9.2	Con FT-Transformer: gradient ascent	20
10	El airbag: la red de seguridad dura	22

11 El sesgo de selección y la exploración	23
11.1 El problema más sutil (y más importante)	23
11.2 La solución: exploración deliberada por ciclo completo	23
12 No-estacionariedad, overfitting y portfolio	24
12.1 No-estacionariedad: el mercado cambia de personalidad	24
12.2 Overfitting: el estudiante que memoriza	24
12.3 Representación del portfolio (Deep Sets)	24
13 Métricas de performance del modelo	26
14 Las dos fases de rollout	27
15 Cómo ejecutar X5 desde cero	28
15.1 El mapa mental	28
15.2 Los 3 modos (+ una casilla)	28
15.3 Paso a paso	28
15.4 Preguntas frecuentes	29
16 Parámetros de configuración de X5	31
17 Glosario completo	32

Capítulo 1

Introducción: qué es X5 y por qué es el cerebro

1.1 El sistema en 30 segundos

Alinvesting es un sistema de trading algorítmico que opera comprando en **soportes** (niveles de precio donde históricamente el precio “rebota”). El sistema tiene varios módulos:

Módulo	Qué hace
X0	Descarga precios y calcula los soportes óptimos.
X1	Coloca y gestiona las órdenes en vivo (buy limits, trailing stop, cierres).
X2	Calcula un score fundamental por activo (¿está sano el activo?).
X3	Calcula features técnicas por activo (RSI, medias móviles, volatilidad...).
X4	Backtester: simula X1 sobre la historia para probar configuraciones.
X5	El cerebro: decide qué parámetros usar según el mercado.

Todos esos módulos dependen de un conjunto de **parámetros de configuración**. Hoy esos parámetros son fijos (los define una persona en `config.py`). El problema es que los parámetros *óptimos no son siempre los mismos*: dependen del estado del mercado.

La idea central

Cuando el mercado cambia de “personalidad” (de alcista a bajista, de calmado a volátil), los parámetros que eran buenos dejan de serlo. **X5 es el módulo que ajusta esos parámetros automáticamente, según cómo está el mercado en cada momento.** Por eso es el cerebro.

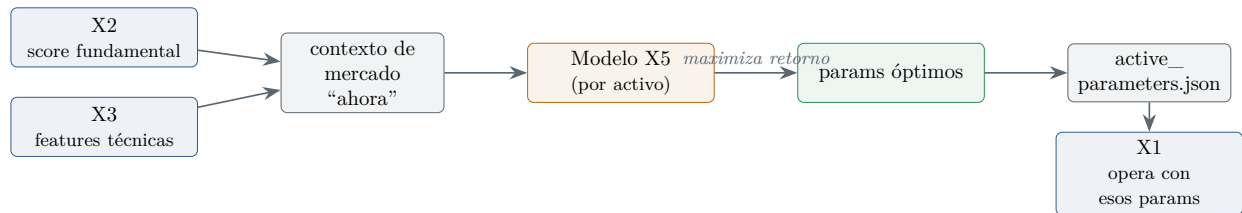
1.2 Los parámetros que X5 controla

Parámetro	Tipo	Qué hace
<code>n_sizes_ejecucion</code>	int	Cuántos soportes activos mantener por activo.
<code>K</code>	float	Peso del aislamiento futuro vs. pasado en el scoring de soportes.
<code>N_EXP</code>	float	Exponente de recencia (cuánto pesan las velas recientes).
<code>LAMBDA</code>	float	Penaliza que los soportes se amontonen en una zona del rango.
<code>A</code>	float	Ganancia mínima (USD) para activar el primer trailing stop.
<code>B</code>	float	Holgura (USD) que el trailing stop mantiene bajo el precio.
<code>LOTAJES_M</code>	int	Multiplicador de lote (tamaño de cada posición).
<code>PERDIDA_MAX</code>	float	Pérdida máxima (USD) por posición antes de cerrarla.

Todo es por activo

X5 no entrena un solo modelo global. Entrena **un modelo independiente por activo** (BTCUSD, ETHUSD, TSLA, GOOGL, NVDA, AMZN). BTC (muy volátil, 24/7) y GOOGL (más estable, horario bursátil) tienen dinámicas distintas, así que sus parámetros óptimos difieren. Cada activo tiene su propio cuaderno de datos, su propio modelo y su propio ritmo de aprendizaje.

1.3 El flujo completo



En una frase: *X5 mira cómo está el mercado (X2 + X3), le pregunta a su modelo qué parámetros darían el mejor retorno, y escribe la respuesta en un archivo que X1 puede leer.*

Capítulo 2

El problema que resuelve X5

2.1 Por qué los parámetros fijos no bastan

Imagina que fijas `LOTAJES_M = 3` (posiciones grandes) porque en un mercado alcista te dio buenos resultados. Llega un *crash*: el precio cae 15% en dos días. Con posiciones grandes, las pérdidas se multiplican. Un valor de `LOTAJES_M = 1` habría sufrido mucho menos.

El problema no es que “3” esté mal. Es que “3” era bueno en un régimen y malo en otro. Ningún valor fijo es óptimo para todos los escenarios.

Ejemplo numérico simple

Supón dos posiciones abiertas en BTC con entrada en \$95.000, y el precio cae a \$88.000 (−7,4%). Con lote base que mueve \$1 por dólar de BTC:

- `LOTAJES_M = 1`: pérdida flotante $\approx 2 \times (88.000 - 95.000) \times 0,01 = -\140 .
- `LOTAJES_M = 3`: pérdida flotante $\approx 2 \times (88.000 - 95.000) \times 0,03 = -\420 .

Mismo mercado, misma estrategia: solo cambiar un parámetro triplica la pérdida. Si X5 hubiera detectado el régimen de caída y bajado `LOTAJES_M` a 1 *antes*, el daño sería un tercio.

2.2 El enfoque: surrogate model + optimización en inferencia

Un **surrogate model** (modelo sustituto) es un modelo que “imita” un sistema costoso de evaluar para poder hacer predicciones baratas. En vez de esperar a que el mercado nos diga si una configuración fue buena (lo cual toma semanas de trading real), entrenamos un modelo que *predice* el resultado.

El plan de X5 tiene dos mitades:

1. **Entrenamiento** — se entrena un modelo que aprende la relación:

$$f(\underbrace{\text{contexto X2+X3}}_{\text{cómo está el mercado}}, \underbrace{\text{parámetros}}_{\text{qué configuración uso}}) \longrightarrow \underbrace{\text{retorno esperado}}_{\text{cuánto gana}}$$

2. **Inferencia** — con el contexto de mercado *fijo* (el de ahora), se busca el conjunto de parámetros que **maximiza** el retorno que el modelo predice. Ese conjunto se escribe en `config/active_parameters.json`.

La inversión clave

El modelo aprende a *predecir* el retorno dado (mercado, parámetros). Pero en inferencia lo usamos “al revés”: fijamos el mercado y **buscamos los parámetros** que producen el retorno más alto. Es optimización sobre las entradas del modelo, no sobre sus pesos.

Capítulo 3

La analogía del chef (el hilo conductor)

A lo largo del documento usaremos una analogía. Vale la pena fijarla bien porque conecta todas las piezas.

En la analogía	En X5
El chef	El modelo de aprendizaje automático.
El cuaderno de recetas	El <i>store</i> de datos (<code>{ACTIVO}_store.csv</code>).
Una receta	Un conjunto de parámetros de configuración.
Cómo estaba la cocina ese día	El contexto de mercado ($X_2 + X_3$).
Cómo quedó el plato	El retorno de la operación (la variable objetivo).
Practicar cocinando	Generar datos con backtesting (<code>--recolectar</code>).
Estudiar el cuaderno	Entrenar el modelo.
Pedir la mejor receta para hoy	Inferir los parámetros (<code>--infer</code>).

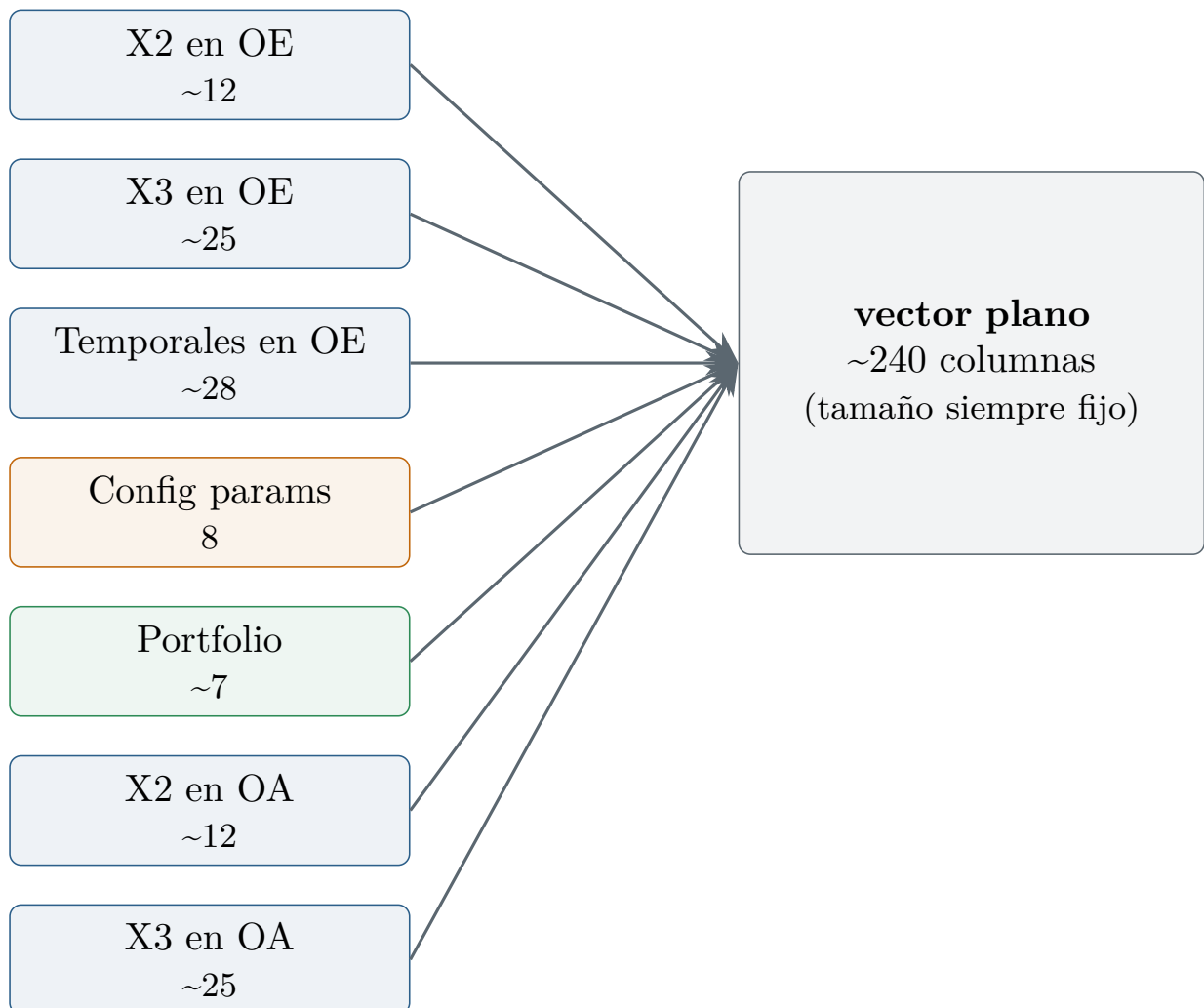
Un chef que cocinó miles de platos en cocinas de distintas condiciones puede predecir: “con esta receta, en una cocina así, el plato sale 8/10”. Al principio, sin embargo, el chef **no sabe nada**: hay que hacerlo cocinar mucho para llenar el cuaderno. Ese es exactamente el estado actual de X5: el cuaderno está vacío, así que el primer trabajo es generar datos.

Capítulo 4

Los datos: qué entra y qué sale del modelo

4.1 Panorama de los inputs (~240 columnas)

Cada fila del cuaderno representa una operación (o un momento del portfolio). Sus columnas se agrupan en cinco bloques, que juntos forman un vector de **tamaño fijo** (~240 números) que entra al modelo:



4.1.1 Bloque 1 y 2 — Contexto de mercado (X2 y X3), en dos momentos

El contexto se captura en **dos instantes distintos** de cada operación:

- **OE (Orden en Espera)** — cuando se coloca el buy limit en un soporte. Es el “ahora” de la decisión; equivale al momento en que en inferencia le preguntamos al modelo.

- **OA (Orden Abierta)** — cuando el precio baja hasta el soporte y la posición se abre. **Entre OE y OA pueden pasar días**, y el mercado puede haber cambiado mucho. Por eso se captura también, con sufijo `_oa`.

¿Y el cierre (OC)?

El contexto del **cierre (OC)** **NO entra al modelo**. Al momento de decidir los parámetros (inferencia) todavía no sabemos cómo va a cerrar la operación: es información del futuro. Las columnas `_oc` se guardan solo para análisis posterior, nunca como entrada.

X2 (fundamentales): responde “¿está el activo fundamentalmente sano?”. Es un score en $[0,1]$ construido con ratios financieros, tendencia histórica y sentimiento de mercado (Fear & Greed). **X3 (técnicas)**: responde “¿cómo se comportó el precio recientemente?”. RSI, medias móviles (SMA/EMA), MACD, ATR (volatilidad), Bandas de Bollinger, momentum, drawdown, distancia a soportes, etc.

4.1.2 Bloque 3 — Features temporales

Derivadas del timestamp, sin datos externos. Capturan estacionalidad:

- `hora` (0–23), `dia_semana` (0=lunes), `dia_mes`, `mes`.
- *Dummies* de día de semana (`ds_lun...ds_dom`) y de mes (`mes_1...mes_12`).
- Proximidad a festivos US: `dias_hasta_festivo`, `dias_desde_festivo` (capados a 30), y banderas `es_vispera_festivo`, `es_post_festivo`.

Se incluyen incluso para crypto: aunque el mercado sea 24/7, los festivos US afectan el volumen institucional.

4.1.3 Bloque 4 — Parámetros de configuración

Los 8 parámetros de la Tabla del Capítulo 1: son “los ingredientes de la receta”. El modelo aprende qué combinación funciona mejor en cada tipo de mercado.

4.1.4 Bloque 5 — Estado del portfolio

Estadísticas del portfolio en ese momento: cuántas posiciones abiertas (`n_ordenes_abiertas`), exposición (`exposicion_usd`), retorno flotante medio y su dispersión (`mean_retorno_pct_abierto`, `std_retorno_pct_abierto`), y el retorno promedio de las últimas 5 operaciones cerradas. En el Capítulo 15 veremos por qué se usan estadísticas resumidas y no las órdenes individuales.

4.2 Los outputs: 3 variables objetivo

El modelo predice **tres** cosas a la vez (Capítulo 8, multi-head):

Nombre en el código	Qué mide	En qué filas se usa
<code>retorno_pct</code>	Retorno % de la operación cerrada.	<code>oc</code>
<code>pnl_flotante_activo</code>	P&L flotante de las posiciones abiertas.	<code>oc</code> + <code>periodico</code>
<code>pnl_cerrado_activo</code>	P&L cerrado acumulado del activo.	<code>oc</code>

La variable que se *maximiza* en inferencia es configurable; por defecto `retorno_pct`, que es la señal más limpia (mide una operación individual, sin ruido de otras posiciones).

Capítulo 5

El store: el cuaderno de recetas

5.1 Dos tipos de fila

El store (`resources/x5/{ACTIVO}_store.csv`) tiene dos tipos de fila, distinguidas por la columna `tipo_registro`:

tipo = oc
una fila por trade cerrado
Y = `retorno_pct`

“¿qué params dan operaciones rentables?”

tipo = periodico
una fila cada 4 velas H1
Y = `pnl_flotante_activo`

“¿qué params mantienen sano el portfolio en rachas malas?”

5.2 Por qué son necesarios los dos tipos

Si el cuaderno solo guardara filas al cerrar operaciones, habría un **hueco peligroso** durante las caídas sostenidas. Veámoslo:

Una racha bajista de 3 días

Momento	Precio BTC	Qué pasa
Lunes 09:00	\$95.000	8 posiciones abiertas, todo normal.
Lunes 15:00	\$92.000	Posiciones en rojo. Ninguna cierra aún.
Martes 10:00	\$88.000	Más rojo. Ninguna cierra.
Miércoles	\$83.000	Recién ahora empiezan a cerrar por <code>PERDIDA_MAX</code> .

Si solo guardáramos filas al cerrar órdenes, durante **3 días de caída el modelo no recibe ningún dato nuevo**. No se entera de que el portfolio estaba sangrando. Cuando llega la primera OC del miércoles, el daño ya está hecho.

La solución son los **registros periódicos**: cada 4 velas H1 (configurable con `X5_FREQ_REGISTRO_PERIODICO`), aunque no cierre ninguna orden, se guarda una fila con el estado actual. Su variable objetivo es `pnl_flotante_activo`, que se vuelve cada vez más negativo durante la caída. Así el modelo aprende: “cuando el RSI cae bajo 30 y el drawdown supera -8% con estos params, el portfolio se sigue deteriorando” $\Rightarrow$ conviene reducir exposición *antes*.

Solo si hay posiciones abiertas

Un registro periódico solo se genera si el activo tiene posiciones abiertas. Si no hay nada expuesto al mercado, no hay `pnl_flotante_activo` que observar y la fila se omite.

5.3 Schema real implementado

Lo que `X4_backtester.py --x5` escribe hoy en cada fila (puede diferir del diseño aspiracional más ambicioso):

- **Identificadores:** `tipo_registro`, `activo`, `timestamp_oe`, `timestamp_oa`, `timestamp_oc`.
- **Config activa (OE):** `n_ejecucion`, `K`, `N_EXP`, `LAMBDA`, `A`, `B`, `LOTAJES_M`, `PERDIDA_MAX`.
- **Temporales, X2, X3 (OE)** y sus versiones **(OA)** con sufijo `_oa`.
- **Portfolio:** `n_ordenes_abiertas`, `n_ordenes_espera`, `exposicion_usd`, `mean_retorno_pct_abierto`, `std_retorno_pct_abierto`, `retorno_promedio_ultimas_5_oc`, `pnl_flotante_activo`.
- **Targets:** `pnl_cerrado_activo`, `retorno_pct`.

Acumulación sin truncar

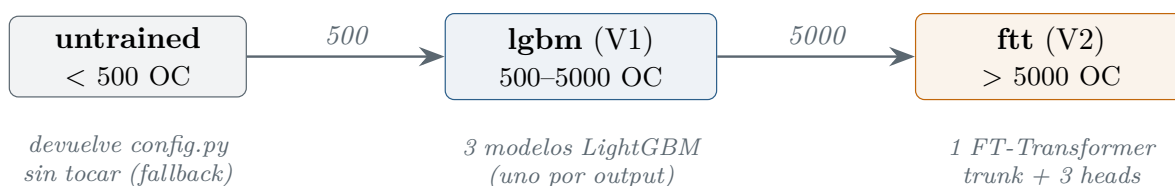
El store acumula **todas** las vueltas del backtesting, incluso las de configuraciones malas. Los registros con params peores son igual de valiosos: le enseñan al modelo qué *no* funciona en cada régimen. Más variedad de (contexto, params, resultado) = más relaciones causa-efecto que el modelo puede aprender.

Capítulo 6

Arquitectura que escala con los datos

6.1 Tres niveles según cuántos datos hay

X5 no arranca con una arquitectura fija. **Sube de nivel automáticamente** según cuántas operaciones cerradas (OC) tenga el store de cada activo:



Los umbrales están en *config.py* (`X5_MIN_TRADES_TRAIN=500`, `X5_MIN_TRADES_FTT=5000`). La transición es **automática y por activo**: BTCUSD puede estar en *ftt* mientras TSLA sigue en *untrained*, dentro de la misma corrida.

Por qué escalar y no empezar directo con la red grande

Un FT-Transformer con pocos datos *sobreajusta* (memoriza en vez de aprender). LightGBM funciona muy bien con datasets pequeños. Empezar simple y crecer con los datos es más robusto que arrancar con la arquitectura más potente y esperar a que “cuaje”.

6.2 Estado actual

El store está **vacío**: los 6 activos están en *untrained*. Mientras un activo esté así, X5 devuelve los parámetros de *config.py* sin modificarlos (un *fallback* seguro: nunca deja que un modelo vacío rompa parámetros que funcionan). El objetivo de la Fase 1 es recolectar datos para pasar a *lgbm*.

Capítulo 7

V1 — Gradient Boosting (LightGBM)

7.1 La intuición: un comité que corrige errores

Gradient Boosting entrena muchos árboles de decisión *en secuencia*, donde cada árbol corrige los errores del anterior. No es una red neuronal. Es el punto de partida más robusto para datos tabulares con pocas filas.

Ejemplo numérico completo

Queremos predecir el retorno de una operación. Tenemos 4 operaciones:

Op	RSI	K	Mercado	Retorno real
A	30	1.0	bajista	+3%
B	75	1.0	alcista	+1%
C	30	0.7	bajista	+4%
D	75	1.5	alcista	-1%

Ronda 0 — predicción inicial. Predice el promedio de todos: $(3 + 1 + 4 - 1)/4 = +1,75\%$ para todos. Los errores (real - predicho) son:

$$e_A = +1,25, \quad e_B = -0,75, \quad e_C = +2,25, \quad e_D = -2,75$$

Ronda 1 — primer árbol. En vez de predecir el retorno, este árbol aprende a predecir los *errores*. Puede descubrir reglas simples como:

$$\text{si RSI} < 50 \rightarrow \text{corregir } +1.5\% \quad \text{si RSI} \geq 50 \text{ y } K > 1.2 \rightarrow \text{corregir } -2\%$$

Aplicando estas correcciones, A sube a +3,25% y D baja a -0,25%: nos acercamos.

Ronda 2 — segundo árbol. Aprende los errores que *aún quedan*. Puede descubrir: “si $K < 0.8 \rightarrow$ corregir un poco más hacia arriba”.

Y así 500 veces. La predicción final es la suma:

$$\text{retorno_predicho} = \underbrace{1,75}_{\text{promedio}} + \text{corr}_1 + \text{corr}_2 + \dots + \text{corr}_{500}$$

Cada árbol es simple. La potencia viene de **acumular cientos de correcciones pequeñas y específicas**. El término “gradient” viene de que las correcciones se calculan usando el gradiente del error, lo que garantiza que cada árbol apunta en la dirección óptima de mejora.

7.2 Cómo se ve en el código

En X5, cuando un activo está en `lgbm`, se entrenan **3 modelos LightGBM independientes** (uno por cada variable objetivo): `retorno`, `flotante`, `cerrado`. Cada uno es un `LGBMRegressor` con ~1000 árboles, *early stopping* sobre un set de validación, y regularización (`reg_alpha`, `reg_lambda`).

Un detalle importante del pipeline

El modelo `flotante` (target `pnl_flotante_activo`) entrena con filas `oc` y `periodico`, porque ese target existe en ambos tipos. Los modelos `retorno` y `cerrado` entrenan solo con filas `oc`. Así los registros periódicos —diseñados para capturar el deterioro en rachas malas— sí alimentan al modelo que más los necesita.

7.3 Ventajas y el único contra

Ventajas	Contra
Funciona muy bien con <50k filas. No requiere normalización. Es interpretable (se ve qué features importaron). Rápido de entrenar.	No es diferenciable $\Rightarrow$ no se puede usar gradient ascent para la inferencia; hay que usar Optuna (Capítulo 9), que es más lento.

Capítulo 8

V2 — FT-Transformer (Feature Tokenizer + Transformer)

Cuando el store supera ~5000 OC, el salto natural es el **FT-Transformer**. Aquí está el corazón conceptual del documento, así que iremos despacio.

8.1 Paso 1 — Feature Tokenizer: cada número se vuelve un vector

En una red neuronal común (un Perceptrón Multicapa, MLP), cada feature entra como un número suelto. El modelo trata todos los números igual, sin saber que “RSI” y “K” son cosas conceptualmente distintas.

El **Feature Tokenizer** convierte cada feature en un **vector** (embedding) propio:

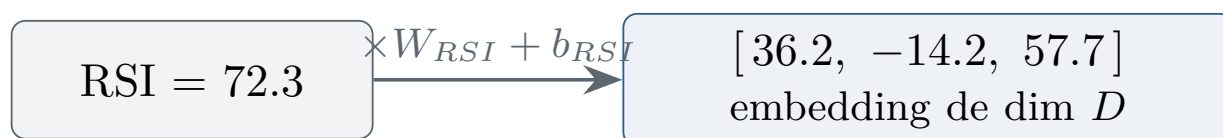
$$\text{embedding}_i = \text{valor}_i \times W_i + b_i$$

donde W_i y b_i son un vector de pesos y un vector de sesgo, *propios de esa feature y aprendidos durante el entrenamiento*.

Con $D = 3$ (en la práctica $D = 192$)

$$\begin{aligned} \text{RSI} &= 72,3, \quad W_{\text{RSI}} = [0,5, -0,2, 0,8], \quad b_{\text{RSI}} = [0,1, 0,3, -0,1] \\ \text{embedding}_{\text{RSI}} &= 72,3 \times [0,5, -0,2, 0,8] + [0,1, 0,3, -0,1] \\ &= [36,15, -14,46, 57,84] + [0,1, 0,3, -0,1] = [36,25, -14,16, 57,74] \end{aligned}$$

La feature K tiene *sus propios* W_K, b_K , distintos. El modelo descubre por sí solo qué transformación es útil para cada feature.



La clave: comparado con una MLP donde RSI entra como el número “72.3”, el FT-Transformer ve un **vector de D números que captura “qué implica que RSI esté en este nivel”** según lo aprendido. RSI=72 “significa” algo distinto que RSI=30, y esa diferencia queda representada.

8.2 Paso 2 — Atención: las features se hablan entre sí

8.2.1 El problema que resuelve la atención

Una MLP aprende la importancia de cada feature *por separado*. Puede aprender “RSI alto es mala señal”, pero no “RSI alto es mala señal *solo cuando* la volatilidad también es alta y $K > 1.2$ ”. Eso requiere capturar la interacción de tres features a la vez. La **atención** hace exactamente eso: deja que cada feature ajuste su representación según el contexto de todas las demás.

8.2.2 La analogía de la reunión de equipo

Imagina un equipo de 5 personas en una reunión. Cada persona es una feature:

RSI: “el precio está sobrecomprado (78)” • K: “el peso futuro es 1.5”
Volatilidad: “el mercado está agitado” • n_sizes: “hay 90 soportes”
Score fundamental: “el activo está sano”

En una MLP, cada persona escribe su conclusión en un papel y la manda a la salida: **no se hablan entre sí**. En un Transformer, **antes de concluir, cada persona escucha a las demás y ajusta lo que va a decir**. RSI piensa: “soy alto, normalmente mala señal...pero oí que la volatilidad también es alta y el fundamental es sano; en ese contexto quizás no sea tan malo”, y ajusta su representación.

8.2.3 El mecanismo concreto: Query, Key, Value

Para que una feature pueda “preguntar” a las demás, el Transformer le asigna tres vectores, derivados de su embedding con matrices aprendidas W_Q, W_K, W_V (compartidas por todas):

$$\begin{aligned}Q_i &= \text{emb}_i W_Q & (\text{“qué busco”}) \\K_i &= \text{emb}_i W_K & (\text{“qué ofrezco”}) \\V_i &= \text{emb}_i W_V & (\text{“lo que comparto”})\end{aligned}$$

RSI le pregunta a las demás

RSI usa su Q_{RSI} y mide su afinidad con cada compañera vía producto interno con los K :

$$Q_{RSI} \cdot K_{\text{Vol}} = 8,3 \quad Q_{RSI} \cdot K_K = 5,1 \quad Q_{RSI} \cdot K_{\text{n_sizes}} = 2,2 \quad Q_{RSI} \cdot K_{\text{fund}} = 4,7$$

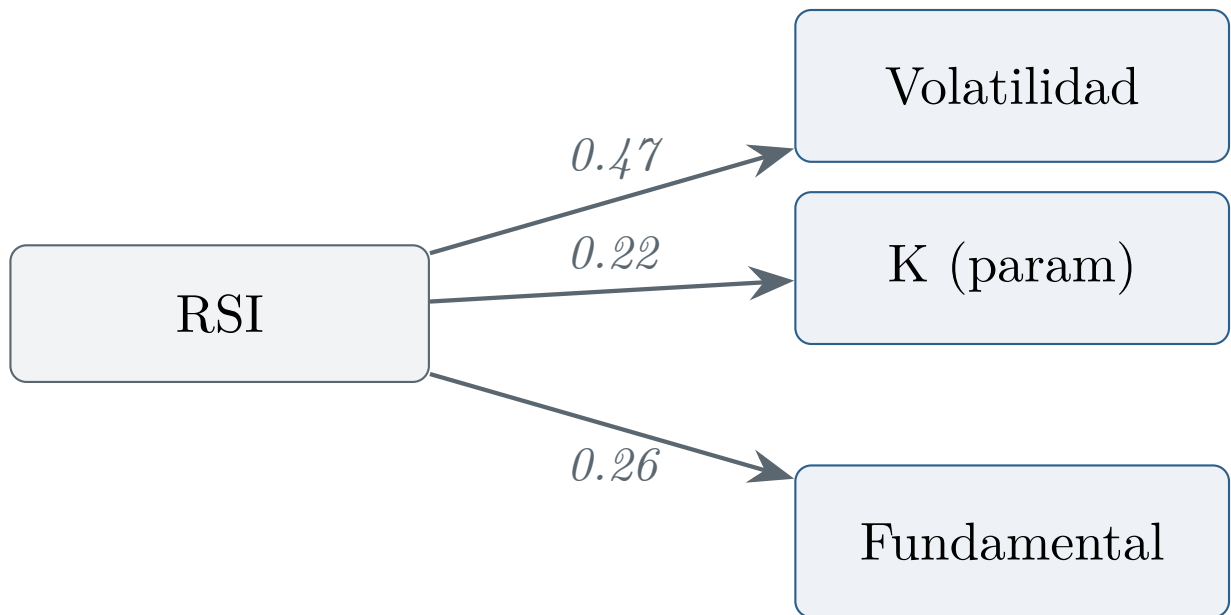
Aplica **softmax** para convertir esos scores en pesos que suman 1:

$$\text{softmax}([8,3, 5,1, 2,2, 4,7]) = [0,47, 0,22, 0,05, 0,26]$$

El nuevo embedding de RSI es la suma ponderada de los *Values*:

$$\text{emb}_{RSI}^{\text{nuevo}} = 0,47 V_{\text{Vol}} + 0,22 V_K + 0,05 V_{\text{n_sizes}} + 0,26 V_{\text{fund}}$$

RSI “absorbió” información de las demás en proporción a cuánto le importaban. Su embedding ya no es “soy RSI = 78”; es “soy RSI = 78 en un contexto de alta volatilidad, fundamental sano y K = 1.5”.



Esto ocurre **simultáneamente para todas las features**: cada una actualiza su representación mirando a las demás.

8.2.4 Multi-head: mirar desde varios ángulos

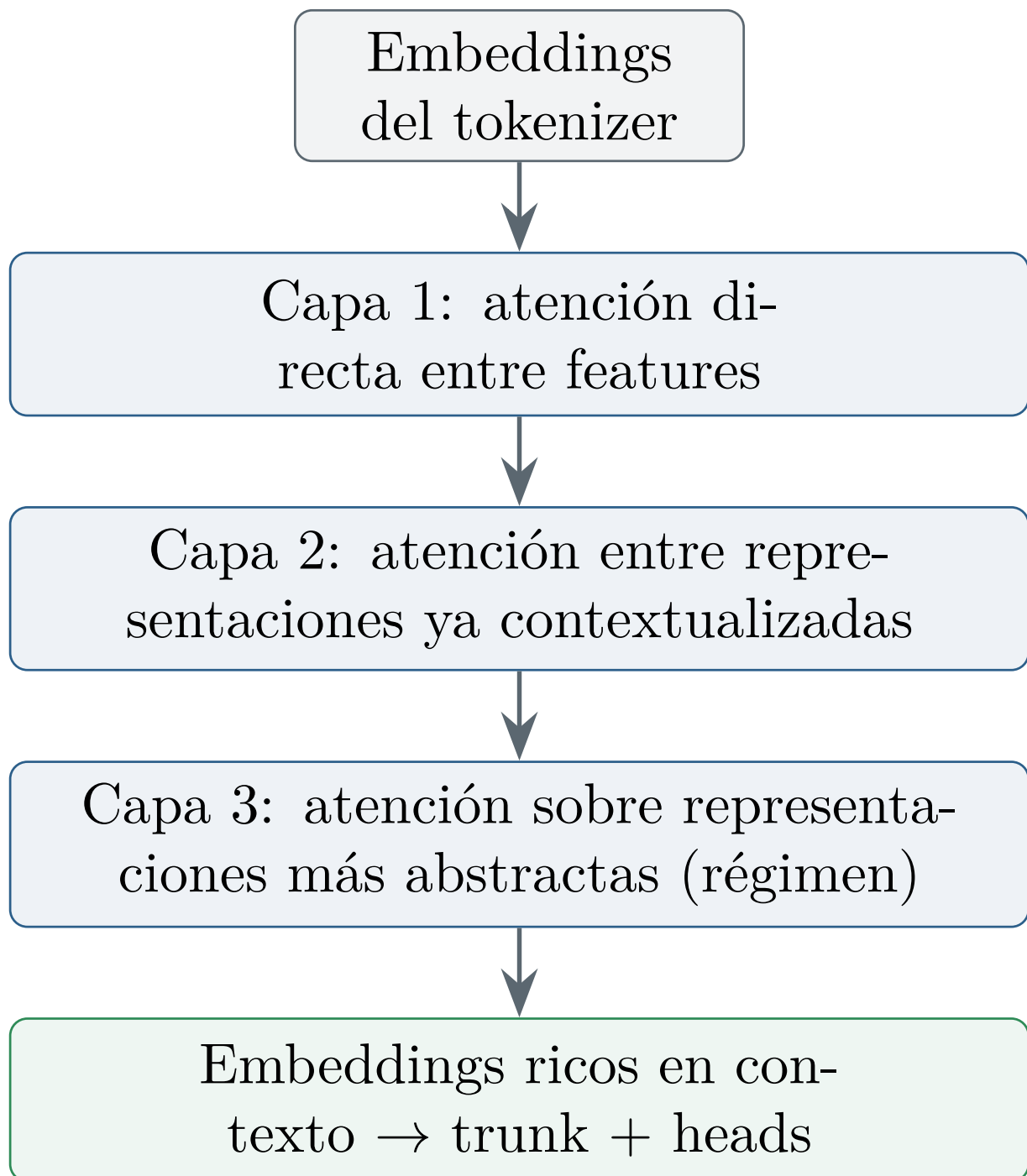
Con una sola “cabeza” de atención, cada feature solo puede mirar a las demás desde un ángulo. La **atención multi-cabeza** (Multi-Head Attention) corre el mecanismo varias veces en paralelo, con matrices W_Q, W_K, W_V distintas por cabeza:

Cabeza 1: relaciones técnicas (RSI $\leftrightarrow$ Volatilidad $\leftrightarrow$ MACD)
Cabeza 2: relaciones params-contexto (K $\leftrightarrow$ tendencia $\leftrightarrow$ n_sizes)
Cabeza 3: relaciones de riesgo (LOTAJES_M $\leftrightarrow$ exposición $\leftrightarrow$ drawdown)

Los resultados de todas las cabezas se concatenan y se proyectan en un vector final. Así el modelo captura múltiples tipos de relación a la vez, sin que una interfiera con otra.

8.3 Paso 3 — Apilar capas: de lo simple a lo abstracto

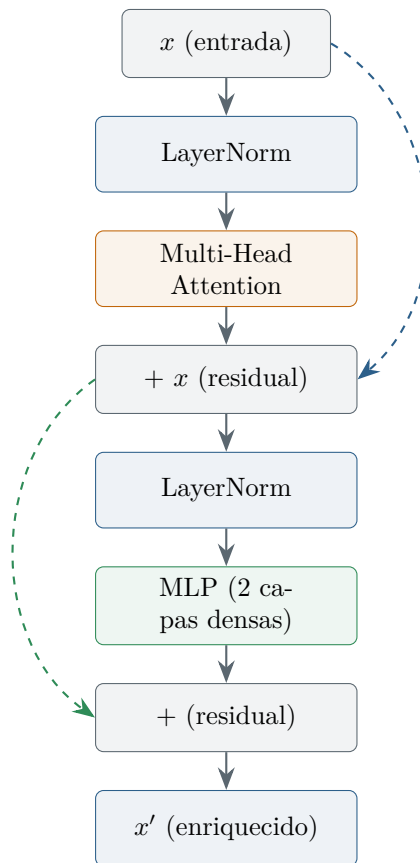
Un solo bloque de atención captura relaciones directas. Pero algunos patrones son más abstractos: “el efecto de K depende de la volatilidad, que a su vez depende del régimen, que se infiere combinando score + MACD + tendencia”. Por eso el Transformer apila varias capas:



Para datos tabulares como los de X5, **2–4 capas** suelen bastar. (Los grandes modelos de lenguaje usan 32–96 capas porque el lenguaje es mucho más complejo.)

8.4 Dentro de cada bloque Transformer

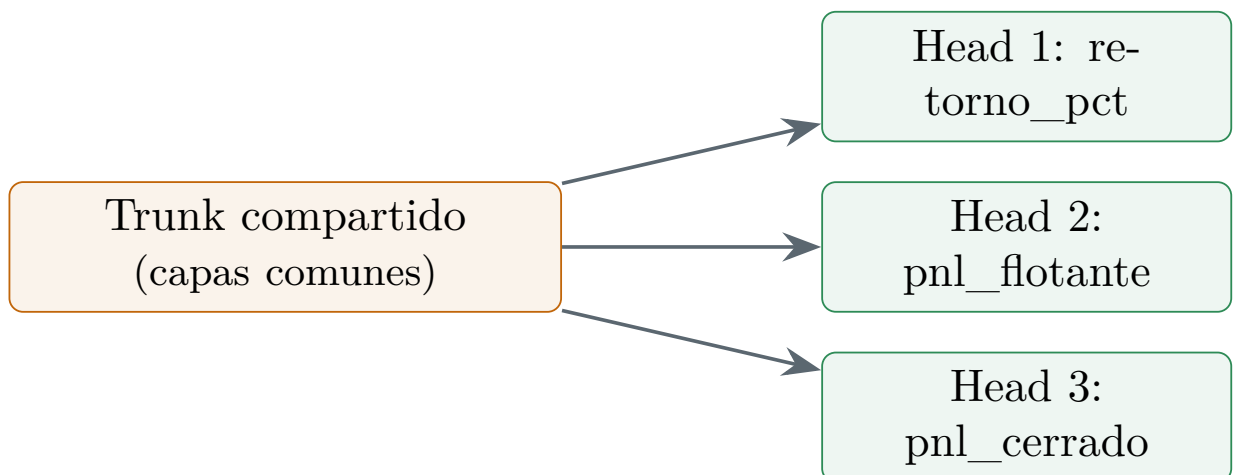
Cada capa no es solo atención pura. El bloque completo (variante *pre-norm*, más estable) es: normalización → atención → conexión residual → normalización → MLP → conexión residual.



- **Conexión residual** (líneas punteadas): se suma la entrada de la capa a su salida. Garantiza que si la capa no aporta nada útil, puede aprender a devolver el input intacto, y el gradiente fluye sin degradarse. Sin esto, apilar más de 2–3 capas es muy difícil de entrenar.
- **LayerNorm**: normaliza el embedding (media 0, desviación 1) antes de cada sub-bloque. Estabiliza el entrenamiento.
- **MLP interno**: tras absorber contexto, cada feature pasa por un pequeño perceptrón que transforma su representación de formas más complejas que la atención sola.

8.5 La salida multi-head: predecir 3 cosas a la vez

Tras las capas Transformer, un **trunk** (tronco) compartido produce una representación general, y tres **heads** (cabezas) predicen cada variable objetivo:



Por qué multi-head es mejor que 3 modelos separados

Como un médico que hace una sola revisión (el trunk) y luego da tres diagnósticos (cardiovascular, respiratorio, metabólico). Compartir el trunk actúa como **regularización**: el modelo no puede sobreajustarse a una señal ruidosa individual si tiene que explicar tres variables a la vez con la misma representación base.

Estado real vs. diseño

En LightGBM las 3 salidas son 3 modelos independientes. La arquitectura multi-head aplica en V2 (FTT). Hoy, en FTT, los 3 heads comparten el tensor de filas oc; incorporar las filas periódicas al head flotante en FTT (como sí ocurre en LightGBM) es un TO DO pendiente.

Capítulo 9

Inferencia: encontrar los parámetros óptimos

Con el contexto de mercado de hoy **fijo**, queremos los params que maximizan el retorno predicho. La estrategia depende del tipo de modelo.

9.1 Con LightGBM: búsqueda con Optuna

LightGBM no es diferenciable, así que no podemos hacer matemática continua sobre él. La estrategia:

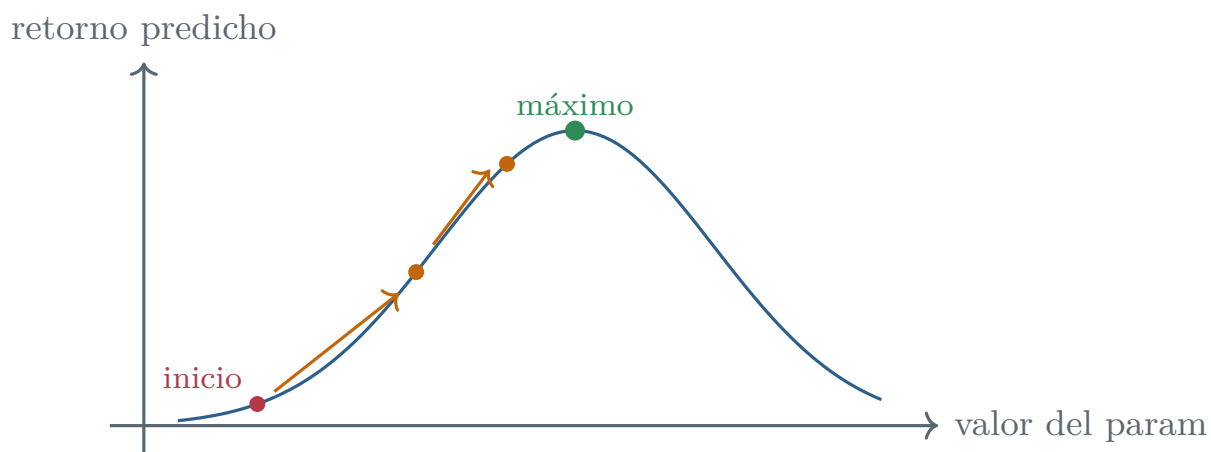
1. Fijar el contexto de mercado actual ($X_2 + X_3$).
2. Probar muchas combinaciones de parámetros.
3. Para cada una, preguntarle al modelo: “¿cuánto predices que gano con estos params?”.
4. Quedarse con la combinación de mayor predicción.

En vez de una rejilla ciega, se usa **Optuna**, un optimizador *bayesiano*: aprende qué zonas del espacio de parámetros son prometedoras y concentra ahí los intentos, en vez de probar todo por igual. Por defecto **200 trials** (`X5_N_OPTUNA_TRIALS`). Es como pedirle al comité de expertos que evalúe 200 recetas y elija la mejor para la cocina de hoy.

9.2 Con FT-Transformer: gradient ascent

Una red neuronal es una función matemática **diferenciable**. Podemos calcular: “si subo K en 0.01, ¿el retorno predicho sube o baja, y cuánto?”. El **gradient ascent** aprovecha eso:

1. Fijar el contexto (no se toca).
2. Inicializar los params en algún punto del espacio.
3. Calcular el gradiente respecto a los *params de entrada* (no a los pesos del modelo).
4. Dar un paso en la dirección que sube el retorno.
5. Repetir hasta llegar a un máximo.



La analogía de la colina con los ojos vendados

Estás en una colina y quieres llegar a la cima, pero con los ojos vendados. En cada paso tocas el suelo con el pie para sentir hacia dónde sube la pendiente y avanzas en esa dirección. El gradient ascent es eso, pero matemático. Es mucho más eficiente que probar 200 combinaciones a ciegas. El contra: puede quedar atrapado en una colina pequeña (máximo local); se mitiga repitiendo desde varios puntos de inicio aleatorios (`X5_ASCENT_RESTARTS`).

Al final, los params se **redondean** (los enteros como `LOTAJES_M`) y se **recortan** a sus rangos válidos.

Capítulo 10

El airbag: la red de seguridad dura

El modelo aprende patrones del pasado. Un crash repentino puede ser tan distinto a todo lo que vio que **no reaccione a tiempo**: recomienda params normales mientras el precio cae 8% en 4 horas. Para ese caso existe el **airbag**: una regla *explícita, no aprendida*, que actúa independientemente de lo que el modelo recomiende.

```
Si en las últimas 4 velas H1 el precio cayó más del umbral:  
    → forzar LOTAJES_M al mínimo (1)  
    → forzar n_sizes_ejecucion al mínimo permitido
```

Los umbrales van en `config.py`: **8% para crypto, 5% para acciones** (`X5_AIRBAG_THRESHOLD`).

El airbag es el último recurso, no el mecanismo principal

La idea es que, con suficiente historia de crashes, el modelo aprenda a recomendar params conservadores *antes* de que el airbag se active. Si el airbag se dispara muy seguido, es señal de que el modelo necesita más datos de ese tipo de escenario. El airbag cubre el riesgo de mercado; el estado `untrained` cubre el riesgo de que el modelo aún no sepa.

Capítulo 11

El sesgo de selección y la exploración

11.1 El problema más sutil (y más importante)

El cuaderno solo contiene operaciones hechas con los params que el sistema consideraba “buenos” en ese momento. **Nunca vemos qué habría pasado con otros params**, porque nunca los probamos.

La analogía del restaurante

Quieres saber cuál es el mejor restaurante de la ciudad. Pero siempre vas al mismo porque te parece bueno. Al final del año tienes 200 reseñas de ese restaurante y cero del resto. No puedes concluir que es el mejor: nunca probaste los otros. El modelo cae en la misma trampa: aprende que “los params actuales son buenos” simplemente porque son los únicos que vio.

11.2 La solución: exploración deliberada por ciclo completo

En el backtesting, una fracción de los ciclos usa params **aleatorios** (exploración) en vez del mejor conocido (explotación):



Un ~30% de exploración (X5_EXPLORATION_RATE) es un punto de partida razonable.

Por qué por ciclo entero y no vela a vela

El backtesting recorre ~2 años de historia por ciclo (con mercados alcistas, bajistas, laterales, crashes). Si aleatorizamos los params *al inicio del ciclo* y los mantenemos fijos, el modelo ve “cómo funcionaron ESTOS params en TODO tipo de mercado”. Puede aprender que $K=1.5$ va bien en bull pero falla en crashes, porque ambos escenarios están en el mismo ciclo. Si aleatorizáramos vela a vela, habría demasiado ruido para atribuir el resultado al contexto.

Los ciclos EXPLORE rinden peor a propósito. Eso **no importa**: su objetivo no es ganar, sino generar datos variados. Son los ciclos más valiosos para que el modelo aprenda qué *no* funciona en cada régimen.

Capítulo 12

No-estacionariedad, overfitting y portfolio

12.1 No-estacionariedad: el mercado cambia de personalidad

Un modelo entrenado en el bull market de 2024 puede ser inútil en el bear market de 2026: los patrones que aprendió ya no aplican. Esto se llama **concept drift**.

Solución — ponderación temporal exponencial: entrenar con todo el historial, pero dando más peso a las operaciones recientes:

$$\text{peso de una operación} = e^{-\lambda \cdot \text{días de antigüedad}}$$

Con λ alto, el modelo “olvida” rápido; con λ bajo, valora más la historia. En X5 esto es `X5_LAMBDA_DECAY` (por defecto 0.001, que da peso $\approx 0,5$ a operaciones de ~ 700 días atrás). Es una sola línea en el `sample_weight` del entrenamiento y no descarta datos valiosos del pasado.

12.2 Overfitting: el estudiante que memoriza

Overfitting = el modelo memoriza los datos de entrenamiento en vez de aprender patrones generalizables. Como un estudiante que memoriza las respuestas exactas de exámenes pasados: saca 100 en ésos, pero falla cuando cambian la redacción. Señales: error muy bajo en train pero alto en datos nuevos; el modelo recomienda siempre los mismos params extremos.

Cómo se mitiga:

1. **Normalizar en %, no en USD:** ganar \$500 en una cuenta grande no es lo mismo que en una chica; en % la señal es comparable.
2. **Dropout:** apagar aleatoriamente algunas neuronas en cada pasada (fuerza redundancia).
3. **Regularización L2:** penaliza pesos muy grandes (evita depender de una sola feature).
4. **Validación temporal:** siempre entrenar con el pasado y validar con el futuro (nunca al revés), para no “filtrar” información futura.
5. **Multi-head:** predecir 3 variables a la vez obliga al trunk a aprender representaciones generales.

12.3 Representación del portfolio (Deep Sets)

Al abrir una nueva orden, el activo puede tener N posiciones abiertas, y N varía (a veces 2, a veces 8). Pero una red neuronal espera entradas de **tamaño fijo**. Hay dos soluciones:

- **V1 — estadísticas resumidas** (la que se usa): en vez de las órdenes individuales, se pasan resúmenes: `n_ordenes_abiertas`, media y desviación del retorno flotante, exposición %. Siempre son los mismos números, sin importar cuántas órdenes haya. Además evita que el modelo aprenda “identidades” espurias de órdenes concretas.
- **V2 — Deep Sets (mean pooling):** cada orden se codifica como un vector pequeño [retorno flotante, horas abierta, distancia entrada, lote], y se *promedian* todos. El promedio es

permutation-invariant: da igual el orden en que se listen las órdenes (el promedio de $[3, 5, 7]$ es el mismo que el de $[7, 3, 5]$). Así el modelo no aprende un sesgo artificial del tipo “la orden #1 siempre importa más”.

Capítulo 13

Métricas de performance del modelo

Cada vez que X5 entrena, calcula y guarda métricas clásicas de Aprendizaje Automático en `resources/x5/Performance/{ACTIVO}_performance.json` (historial, con *split* 80/20 cronológico, no aleatorio, para respetar el orden temporal). Para cada target y cada set (train/test):

Métrica	Fórmula	Qué mide
MAE	$\text{mean}(y - \hat{y})$	Error absoluto promedio, en las unidades del target.
MSE	$\text{mean}((y - \hat{y})^2)$	Penaliza los errores grandes cuadráticamente.
MAPE	$100 \cdot \text{mean}(y - \hat{y} / y)$	Error porcentual relativo. None si $ y \approx 0$.
R^2	$1 - \text{SS}_{\text{res}}/\text{SS}_{\text{tot}}$	% de varianza explicada. 1 = perfecto, 0 = predecir la media, negativo = peor que la media.

Señales de alerta

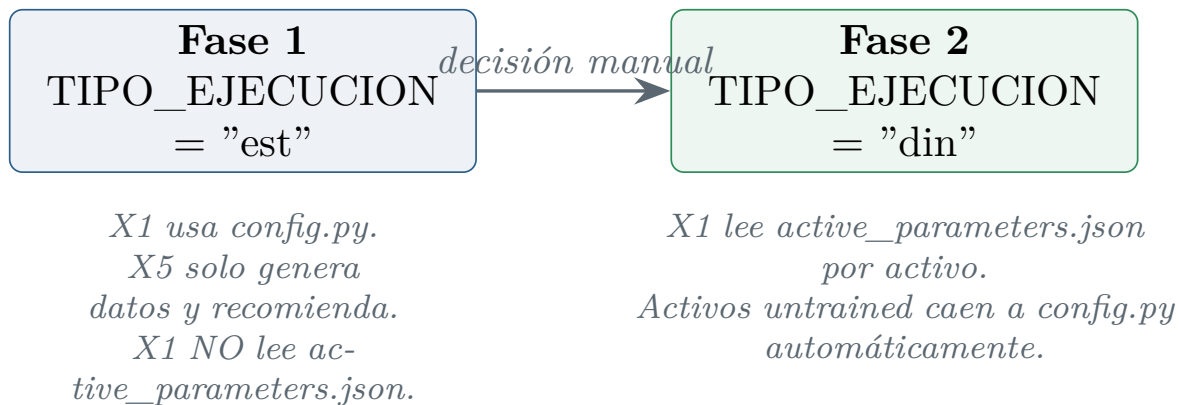
- $R^2_{\text{train}} \gg R^2_{\text{test}}$: overfitting claro.
- $R^2_{\text{test}} < 0$: el modelo es peor que predecir la media (store con mucho ruido o poca variedad de contextos).
- MAE test creciente entre entrenamientos: el modelo se está degradando.
- MAPE **None** frecuente en **flotante/cerrado**: normal (targets cercanos a 0); usar MAE y R^2 para esos heads.

La evolución de estas métricas es la fuente principal para decidir cuándo pasar de V1 a V2 y para calibrar los umbrales.

Capítulo 14

Las dos fases de rollout

X5 se integra en dos fases. El interruptor es TIPO_EJECUCION en `config.py`.



model_status	Significado	Qué usa X1 (en Fase 2)
untrained	<500 OC, sin datos.	<code>config.py</code> (fallback).
lgbm	Modelo V1 activo.	params de <code>active_parameters.json</code> .
ftt	Modelo V2 activo.	params de <code>active_parameters.json</code> .

La transición es **por activo**: BTCUSD puede estar en `din` usando su modelo mientras TSLA sigue cayendo a `config.py`, todo en la misma corrida. Cambiar a Fase 2 es la única acción manual, y se hace cuando las métricas dan confianza.

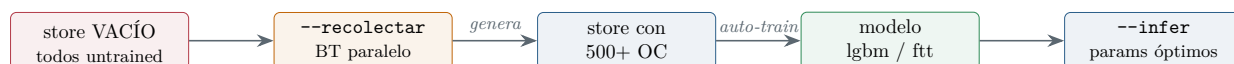
El ciclo de retroalimentación cerrado (objetivo final)

En Fase 2, X1 opera con params dinámicos y genera trades reales, que alimentan el store, que reentrena el modelo, que mejora los params que usa X1. El sistema se corrige solo. Los backtesters dejan de ser la fuente principal de datos a medida que X1 en vivo acumula operaciones reales con variedad de contextos.

Capítulo 15

Cómo ejecutar X5 desde cero

15.1 El mapa mental



15.2 Los 3 modos (+ una casilla)

X5 se simplificó de 5 modos a 3 + una casilla. `--vela` fue eliminado (era un alias exacto de `--infer`).

```
python scripts/X5_macro_brain.py --recolectar --version V1 # generar datos + entrenar
python scripts/X5_macro_brain.py --infer                  # recomendar params
python scripts/X5_macro_brain.py --infer --train          # reentrenar y recomendar
python scripts/X5_macro_brain.py --status                 # diagnostico
python scripts/X5_macro_brain.py --infer --activo BTCUSD  # solo un activo (merge)
```

15.3 Paso a paso

Paso 0 — Requisitos. Datos en `Data/` y `Data_minuto/` (los genera X0), scores en `resources/x2/`, y las dependencias `lightgbm`, `torch`, `optuna`.

Paso 1 — Ver el estado.

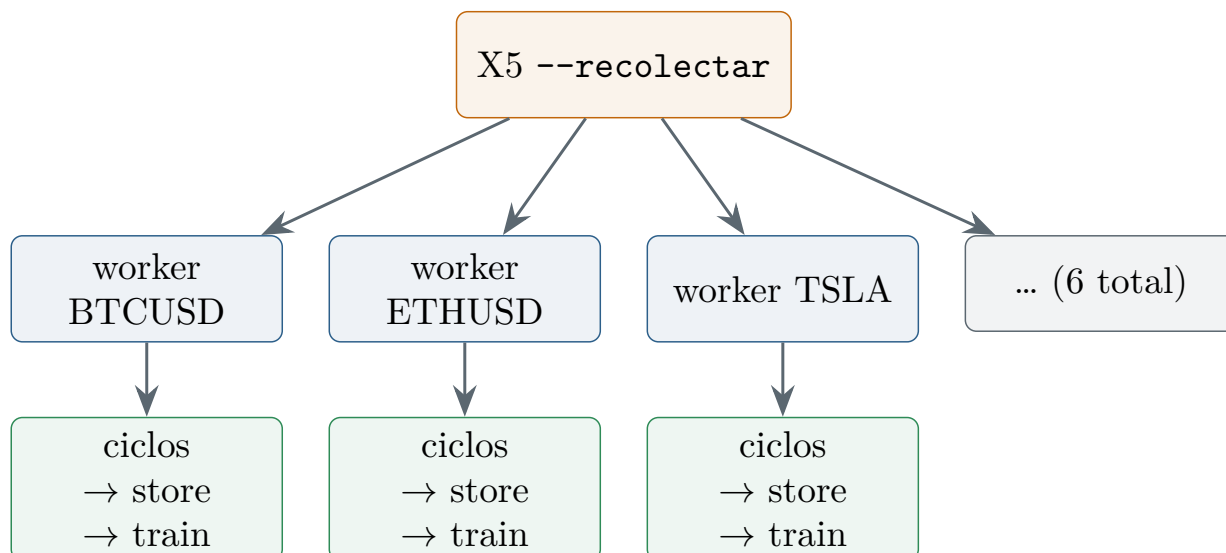
```
python scripts/X5_macro_brain.py --status
```

Al inicio: 0 OC y untrained en los 6 activos. Es lo esperado.

Paso 2 — Recolectar (el paso principal y más largo).

```
python scripts/X5_macro_brain.py --recolectar --version V1
```

Qué pasa por dentro:



- X5 lanza **6 workers en paralelo**, uno por activo (como 6 chefs cocinando cada uno en su cocina).
- Cada worker corre un **ciclo** = un backtest completo desde `fecha_inicio`, simulando X1 vela a vela y anotando filas en su cuaderno.
- Al terminar el ciclo, **reinicia desde `fecha_inicio`** con otros params (explore/exploit), y así hasta `X5_N_CICLOS_BT` ciclos o `X5_MIN_TRADES_FTT` OC.
- **Apenas un activo cruza 500 OC**, ese worker **entrena su modelo** sin esperar a los demás. Desde ahí sus ciclos EXPLOIT usan el modelo recién entrenado: *los params se van corrigiendo por activo*.
- **Aislamiento**: cada worker usa su carpeta `resources_{V}_{ACTIVO}` para el checkpoint/equity de X4; el cuaderno ya es por activo. Así 6 procesos no se pisan.

El primer ciclo es lento

El primer ciclo de cada activo hace *cold start*: recalcula soportes desde cero (varios minutos). Los ciclos siguientes reutilizan la cache de soportes de ese activo.

Paso 3 — Confirmar modelos. Repetir `--status`: deberías ver activos en `lgbm` (500–5000 OC) o `ftt` (>5000). Las métricas quedan en `resources/x5/Performance/`.

Paso 4 — Recomendar params.

```
python scripts/X5_macro_brain.py --infer
```

Lee el contexto actual, busca los params que maximizan `retorno_pct` y los escribe en `config/active_parameters.json`. Los activos aún `untrained` salen con sus valores de `config.py`.

Paso 5 — Activar Fase 2 (cuando confíes en las métricas): en `config.py` poner `TIPO_EJECUCION = "din"`.

15.4 Preguntas frecuentes

¿Tengo que llamar `--train` aparte? No. `--recolectar` ya entrena solo cuando cada activo cruza el umbral. `--train` es solo la casilla de `--infer` para forzar un reentrenamiento puntual.

¿`--recolectar` borra lo anterior? No borra el cuaderno (acumula). Cada ciclo de backtest sí parte de cero en la simulación, pero las filas se *agregan*.

¿Puedo cortar y retomar? Sí. El store persiste en disco; al relanzar, cada worker sigue sumando sobre lo que ya había.

¿Por qué en paralelo? Para que los 6 activos avancen a la vez y cada uno corrija sus params apenas tenga datos, en vez de esperar a que termine uno para empezar el otro.

Capítulo 16

Parámetros de configuración de X5

Todos en `config.py`, bajo el bloque de X5:

Parámetro	Valor	Qué controla
TIPO_EJECUCION	"est"	Fase 1 (est) o Fase 2 (din).
X5_EXPLORATION_RATE	0.30	Fracción de ciclos con params aleatorios.
X5_FREQ_REGISTRO_PERIODICO	4	Snapshot periódico cada N velas H1.
X5_N_CICLOS_BT	20	Máx. de ciclos por activo en <code>--recolectar</code> .
X5_MIN_TRADES_TRAIN	500	OC mínimas para entrenar (bajo esto: untrained).
X5_MIN_TRADES_FTT	5000	OC para pasar de LightGBM a FT-Transformer.
X5_RETRAIN_EVERY_N_VELAS	48	Reentrenamiento completo (modo LightGBM).
X5_WINDOW_TRAIN	None	Ventana de entrenamiento (None = todo el store).
X5_LAMBDA_DECAY	0.001	Decaimiento del peso temporal.
X5_N_OPTUNA_TRIALS	200	Intentos de Optuna por inferencia (LightGBM).
X5_ASCENT_RESTARTS	10	Puntos de inicio del gradient ascent (FTT).
X5_ASCENT_STEPS	300	Pasos de ascent por inicio.
X5_ASCENT_LR	0.01	Tamaño del paso del ascent.
X5_AIRBAG_THRESHOLD	8% / 5%	Caída que dispara el airbag (crypto / acciones).
X5_N_MINIMO	30 / 20	N mínimo al activar el airbag.
X5_PARAM_RANGES	—	Rangos de búsqueda de cada param, por activo.

Capítulo 17

Glosario completo

Surrogate model Modelo que imita el sistema real para predecir barato el resultado de una operación.

Tabular data Datos en filas y columnas (como Excel). Cada fila una operación, cada columna una feature.

Feature Una columna de entrada del modelo (RSI, K, n_ordenes_abiertas...).

Embedding Representar un valor como un vector, para procesarlo con más riqueza.

Atención (attention) Mecanismo que deja que cada feature ajuste su representación mirando a las demás. Base de los Transformers.

Q / K / V Query (qué busco), Key (qué ofrezco), Value (lo que comparto). La atención es una suma ponderada de Values, con pesos del producto $Q \cdot K$.

Softmax Convierte números arbitrarios en pesos que suman 1.

Multi-Head Attention Correr la atención varias veces en paralelo, cada cabeza capturando un tipo de relación.

Conexión residual Sumar la entrada de una capa a su salida ($x' = \text{capa}(x) + x$). Deja fluir el gradiente en redes profundas.

Layer Norm Normaliza el embedding (media 0, desv. 1) antes de cada sub-bloque. Estabiliza el entrenamiento.

Transformer Arquitectura basada en atención. El FT-Transformer la adapta a datos tabulares.

FT-Transformer Feature Tokenizer + Transformer. Convierte cada feature en embedding y aplica atención entre ellas.

Gradient Boosting Muchos árboles en secuencia; cada uno corrige el error del anterior. LightGBM lo implementa.

LightGBM Implementación rápida de Gradient Boosting. V1 de X5.

Multi-head (salida) Trunk compartido + varias cabezas, cada una predice una variable.

Trunk / backbone Capas internas compartidas de una red multi-head.

Inferencia Usar el modelo entrenado para predecir. En X5: hallar los params óptimos.

Gradient ascent Maximizar una función avanzando en la dirección de su gradiente. Se optimizan los inputs de params, no los pesos.

Overfitting El modelo memoriza el pasado en vez de aprender patrones generales.

Dropout Apagar neuronas al azar durante el entrenamiento. Previene overfitting.

Regularización L2 Penaliza pesos grandes; evita depender de una feature.

Concept drift El patrón aprendido deja de valer porque el mundo cambió (bull $\rightarrow$ bear).

Permutation-invariant El resultado no cambia al reordenar las entradas (un promedio).

Deep Sets / mean pooling Procesar conjuntos de tamaño variable promediando embeddings.

OE / OA / OC Orden en Espera / Orden Abierta / Orden Cerrada.

retorno__pct Retorno porcentual de una operación. Variable objetivo principal.

Registro periódico Fila del store cada N velas H1 aunque no haya OC. Captura el deterioro del portfolio.

pnl_flotante_activo P&L no realizado de las posiciones abiertas. Target de los registros periódicos.

model_status Estado por activo: `untrained` / `lgbm` / `ftt`.

Airbag Regla dura (no aprendida) que fuerza params mínimos ante una caída fuerte.

Aprendizaje online / batch Online: un paso de gradiente por dato nuevo (visión Fase 2).
Batch: reentrenar sobre todo el store acumulado (lo actual).